

ELMAR
CNC-Cutter

Autoren: Alexandra Roth 7025637
David Atrops 7013979
Dennis Smirnow 7023434
Leon Ronge 7026641
Samuel Drees 7026354

Studiengang: Maschinenbau und Design

Erstprüfer: Prof. Dr. Elmar Wings
Malena Wilken

Abgabedatum: 9. Mai 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	5
 I. Domain Knowledge - Application	 7
 II. Domain Knowledge - Hardware	 9
1. Sensor: Endschalter	11
1.1. Einleitung: Positionssensoren	11
1.2. Definition und Funktion: Elektromechanischer Endschalter	11
1.3. Specification	11
1.4. Verwendung und Schaltfunktionen	12
1.5. Analyse: Vor- und Nachteile	13
1.6. Ausblick: Intelligente und vernetzte Sensorik	14
1.7. Library	14
1.8. Beispiel	14
1.8.1. Manual	14
1.8.2. Erklärung des Beispiel-Codes	14
1.9. Justage	15
1.10. Tests	15
1.10.1. Einfacher Funktions-Test	15
1.10.2. Manual	16
1.10.3. Einfacher Test-Code	16
1.11. Einfache Anwendung	16
1.11.1. Manual	17
1.12. Further Readings	18
 2. Druckknopf	 19
2.1. General	19
2.1.1. Functions	19
2.2. Example	19
2.2.1. Manual	19
2.2.2. Code	19
2.2.3. Files	19
2.3. Einfacher Testcode	19
2.3.1. Manual	19
2.3.2. Code	19
2.4. Einfache Anwendung	19
2.4.1. Manual	19
2.4.2. Code	19
2.5. Further Readings	19
 III. Domain Knowledge - Tools	 21

IV. Developmenmt	23
V. Monitoring	25
VI. Verification - Evaluation - Conclusion	27
VII. Anhang	29
Literaturverzeichnis	31
Literaturverzeichnis	31

Abbildungsverzeichnis

1.1. Darstellung des im Projekt verwendeten Endschalters	12
1.2. Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO	13

Teil I.

Domain Knowledge - Application

Teil II.

Domain Knowledge - Hardware

1. Sensor: Endschalter

1.1. Einleitung: Positionssensoren

In der industriellen Fertigungssteuerung nimmt die Erfassung von Zuständen eine zentrale Rolle ein. Gemäß der Fachliteratur wandelt ein Sensor (lat. sensus, der Sinn) „eine physikalische Größe (z. B. Kraft oder Temperatur) mit Hilfe eines physikalischen Effekts in ein elektrisches Signal um“ [Her+12, S. 433]. Speziell im Maschinenbau wird bei der Steuerung zwischen zwei grundlegenden Messverfahren unterschieden: Der „Positionserfassung durch Schalter“ und der „Positionserfassung durch Wegbeobachtung“. [Her+12, S. 435]

Bevor der Fokus auf kontaktbehaftete Schalter gelegt wird, sind folgende berührungslose Sensoren (Sensorschalter) als moderner Industriestandard zu nennen:

- Induktive Sensoren: Erfassen metallische Elemente „ohne Kontakt zwischen der Bewegung und dem Sensor“ [Her+12, S. 436].
- Kapazitive Sensoren: Nutzen die Beeinflussung eines elektrischen Feldes zur Kapazitätsänderung [Her+12, S. 438].
- Optische Sensoren: Arbeiten mit Infrarot-Lichtstrahlen zur Objekterkennung (z. B. Lichttaster oder Lichtschranken) [Her+12, S. 440].
- Ultraschall-Sensoren: Basieren auf der Messung der Laufzeit von Schallimpulsen [Her+12, S. 443].

1.2. Definition und Funktion: Elektromechanischer Endschalter

Ein elektromechanischer Endschalter ist ein kontaktbehafteter Positionssensor. Ein solcher Schalter „hat die Aufgabe, das Erreichen einer bestimmten Position (Endlage) zu melden oder einen Schaltvorgang auszulösen“ [Her+12, S. 435]. Dabei wird durch das Betätigen eines mechanischen Elements, wie beispielsweise eines Hebels oder Tasters, ein elektrischer Kontakt geöffnet oder geschlossen [Her+12, S. 120–122].

Technische Spezifikationen (Beispiel Creality): Bei dem vorliegenden Bauteil handelt es sich um einen „originalen Endschalter von Creality 3D, der bei fast allen FDM Druckern von Creality 3D verbaut ist“ [3dj].

1.3. Specification

- Kompatibilität: Geeignet für die X-, Y- oder Z-Achse von Modellen wie der Ender-3 oder CR-10 Serie [3dj][Bas].
- Hersteller: Creality
- Hersteller SKU: 4004180004
- Gewicht: 7.3g
- Abmaße: 28mm x 20mm x 9mm

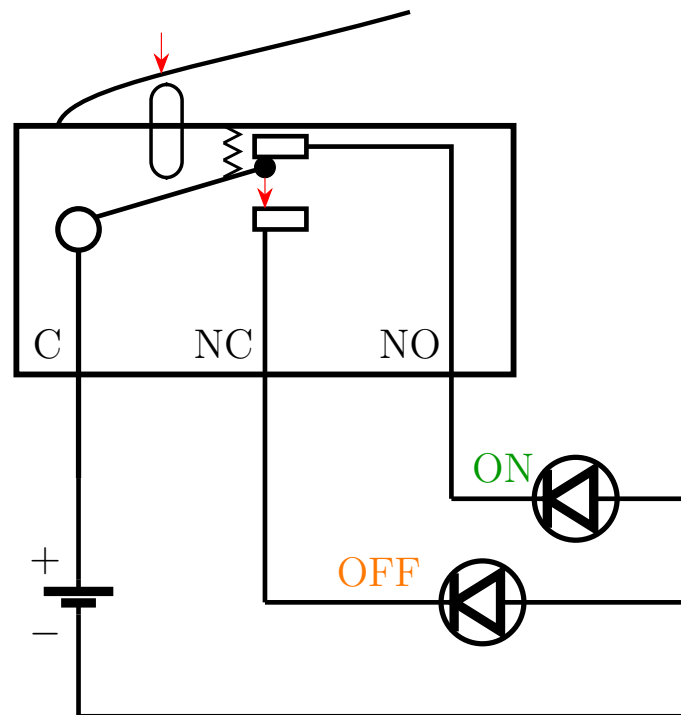


Abbildung 1.2.: Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO

Haupteinsatzgebiete:

- Endstopp-Erkennung: Schutz der Maschine vor dem Überfahren der mechanischen Grenzen.
- Referenzschalter (Homing): Der Schalter dient als „Referenzschalter“, um den Nullpunkt einer Achse (z. B. beim 3D-Drucker) zu kalibrieren [Bas].

1.5. Analyse: Vor- und Nachteile

Vorteile:

- Wirtschaftlichkeit: Sehr kostengünstig (ca. 4-5 € pro Stück) [3DJake, Bastelgarage].
- Einfachheit: Direkte Erzeugung eines binären Signals ohne aufwändige Auswerteelektronik.
- Robustheit gegen EMV: Unempfindlich gegenüber elektromagnetischen Störungen, da kein Oszillator wie bei induktiven Sensoren benötigt wird.

Nachteile:

- Mechanischer Verschleiß: Durch den physischen Kontakt unterliegen die Bauteile einer Abnutzung.
- Geschwindigkeit: Mechanische Kontakte können „prellen“ und sind langsamer als elektronische Lösungen [Her+12, S. 444].
- Technologische Verbreitung: Obwohl immer noch sehr viele mechanische Endschalter eingesetzt werden, ist der berührungslose Endschalter (Sensorschalter) in nahezu allen Bereichen als Standard-Bauelement zu finden [Her+12, S. 435].

1.6. Ausblick: Intelligente und vernetzte Sensorik

Die Zukunft der Positionserfassung liegt in der bidirektionalen Kommunikation. Ein moderner Standard ist hierbei IO-Link, welcher eine „feldbusunabhängige Kommunikation zwischen Sensoren und Aktoren und der Maschinensteuerung“ beschreibt [Her+12, S. 510].

Während klassische Endschalter nur „1 Bit“ an Informationen liefern (Ein/Aus), können intelligente Systeme über IO-Link zusätzlich Servicedaten übertragen, „die zur Einstellung oder Diagnose eines Devices führen“ [Her+12, S. 512].

Dies ermöglicht eine „vorbeugende Instandhaltung“, indem beispielsweise Verschleiß gemeldet wird, bevor es zum Maschinenausfall kommt [Her+12, S. 471, 512].

1.7. Library

Für die Verwendung des Endschalters werden keine zusätzlichen Bibliotheken benötigt. Es werden Funktionen aus der Arduino-Standardbibliothek verwendet:

- `pinMode()`: Konfiguriert einen Pin als Ein- oder Ausgang.
- `digitalRead()`: Liest den aktuellen Zustand eines digitalen Eingangs.
- `digitalWrite()`: Setzt einen digitalen Ausgang auf HIGH oder LOW.
- `Serial.begin()`: Initialisiert die serielle Kommunikation.
- `Serial.print()` / `Serial.println()`: Gibt Daten im Serial Monitor aus.
- `delay()`: Erzeugt eine zeitliche Verzögerung im Programmablauf.

1.8. Beispiel

Dieses Beispiel zeigt, wie der Endschalter über einen digitalen Eingang ausgelesen werden kann. Dabei wird der interne Pull-Up-Widerstand verwendet. Der Zustand des Schalters wird im Serial Monitor angezeigt und ändert sich beim Betätigen von HIGH auf LOW.

1.8.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND.

1.8.2. Erklärung des Beispiel-Codes

Nach der Initialisierung durch die Funktion `setup()` wird das Programm durch die Funktion `loop()` kontinuierlich ausgeführt. Diese Funktion stellt eine Endlosschleife dar, in der der Zustand des Endschalters bei jedem Durchlauf überprüft wird.[Ard; Arde]

Der verwendete Pin wird mit der Funktion `pinMode()` als Eingang oder Ausgang konfiguriert. [Ard] Dabei wird der Modus `INPUT_PULLUP` verwendet, um den internen Pull-Up-Widerstand zu aktivieren.[Arda] Der interne Pull-Up-Widerstand sorgt dafür, dass am Eingang im Ruhezustand ein definiertes HIGH-Signal anliegt. Innerhalb der `loop()`-Funktion wird der Zustand des Endschalters mit der Funktion `digitalRead()` ausgelesen. Der gelesene Wert wird in einer Variablen gespeichert und über die serielle Schnittstelle ausgegeben.[Ardd] Die Funktionen `Serial.print()` und `Serial.println()` werden verwendet, um den aktuellen Zustand des Eingangs im Serial

Monitor darzustellen.[Ardb] Durch die Verwendung des internen Pull-Up-Widerstands ergibt sich eine invertierte Logik. Im nicht betätigten Zustand des Endschalters liegt ein HIGH-Signal am Eingang an. Wird der Endschalter betätigt liegt ein LOW-Signal an. Zur besseren Lesbarkeit der Ausgabe wird eine Verzögerung mit der Funktion `delay()` erzeugt.[Arde]

Listing 1.1.: Einfaches Beispiel zum Lesen eines mechanischen Endschalters

```

1000 /**
      * @file MicroSwitch_BasicRead.ino
1002  * @brief Basic example for reading a micro switch.
      */
1004 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
      switch. */

1006 /**
      * @brief Initializes the serial interface and the input pin.
1008  */
      void setup()
1010 {
      Serial.begin(115200);
1012   pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
      }

1014 /**
      * @brief Reads the switch state and prints it to the serial monitor.
1016  */
      void loop()
1018 {
1020   int switchState = digitalRead(MICRO_SWITCH_PIN);

1022   Serial.print("Micro switch state: ");
      Serial.println(switchState);

1024   delay(200);
1026 }

```

../Code/MicroSwitchBasicRead/MicroSwitchBasicRead.ino

1.9. Justage

Die Justage (auch "Justierung") ist eine "Tätigkeit, welche ein Messgerät in einen gebrauchstauglichen Betriebszustand versetzt. Sie kann manuell, halb automatisch oder automatisch erfolgen." [Hel, S. 17] Sie kann auch als das "abgleichen des Nullpunktes und der Kennliniensteigung" [Ber24, S. 17] verstanden werden. Da der verwendete Endschalter nur die analogen Werte OPEN = 0 und CLOSED = 1 detektieren kann, sodass keine Kennlinie im herkömmlichen Sinne vorliegt, liegt die anwendungsspezifische Justage-Aufgabe darin, den gesamten Endschalter physisch in seiner Position am Rahmengestell so auszurichten, dass er die entsprechende bewegliche Achse im Rahmen einer Referenzfahrt in ihrer Bewegung eingrenzt, aber gleichzeitig im Normalbetrieb den erforderlichen Bauraum nicht künstlich einschränkt.

1.10. Tests

1.10.1. Einfacher Funktions-Test

Im einfachsten Anwendungstest wird der Endschalter verwendet, um eine LED zu steuern. Hierbei wird der Zustand des Schalters ausgewertet und auf einen Ausgang übertragen. Wird der Endschalter betätigt, schaltet sich die LED ein. Im nicht betä-

tigten Zustand bleibt die LED ausgeschaltet. Durch diesen Test kann die Funktion des Sensors sowie die Verarbeitung des Signals im System überprüft werden.

1.10.2. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND. Die LED wird an einen digitalen Ausgang (z. B. D13) angeschlossen.

Beim Betätigen des Endschalters wird die LED eingeschaltet, im nicht betätigten Zustand bleibt sie ausgeschaltet.

1.10.3. Einfacher Test-Code

Die Funktion `loop()` läuft dauerhaft und liest den Zustand des Mikroschalters ein. Da `INPUT_PULLUP` verwendet wird, bedeutet `LOW` = gedrückt. Ist der Schalter gedrückt, wird die LED eingeschaltet, sonst ausgeschaltet.

Listing 1.2.: An- und Ausschalten einer LED mittels Endschalter

```

1000 /**
1001  * @file MicroSwitch_LEDEExample.ino
1002  * @brief Simple application example using a micro switch and an LED.
1003  */
1004
1005 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1006     switch. */
1007 const int LED_PIN = 13;          /**< Digital output pin for the built-in
1008     LED. */
1009
1010 /**
1011  * @brief Initializes the input and output pins.
1012  */
1013 void setup()
1014 {
1015     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1016     pinMode(LED_PIN, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the LED depending on the micro switch state.
1021  */
1022 void loop()
1023 {
1024     int switchState = digitalRead(MICRO_SWITCH_PIN);
1025
1026     if (switchState == LOW)
1027     {
1028         digitalWrite(LED_PIN, HIGH);
1029     }
1030     else
1031     {
1032         digitalWrite(LED_PIN, LOW);
1033     }
1034 }

```

../Code/MicroSwitchLEDEExample/MicroSwitchLEDEExample.ino

1.11. Einfache Anwendung

Das Programm führt eine Referenzfahrt mit einem Schrittmotor aus.

Im `setup()` werden die Pins für den Treiber und den Endschalter eingerichtet. Danach wird der Motortreiber aktiviert und die Referenzfahrt gestartet.

Die Funktion `oneStep()` erzeugt einen einzelnen Schritt des Motors.

Mit `moveSteps()` kann der Motor eine festgelegte Anzahl an Schritten in eine bestimmte Richtung fahren.

In `homeAxis()` fährt der Motor so lange, bis der Endschalter gedrückt wird. Danach wartet das Programm kurz und fährt den Motor einige Schritte zurück.

1.11.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters wird an D2 angeschlossen, der zweite Anschluss liegt auf GND. Der Schrittmotortreiber wird über die Steuerpins STEP und DIR an digitale Ausgänge des Mikrocontrollers angeschlossen (z. B. STEP an D3 und DIR an D4).

Listing 1.3.: Referenzfahrt einer Achse mit Endschalter

```

1000 /**
      * @file referenzfahrt.ino
1002  * @brief Homing routine using a stepper motor, A4988 driver, and
      *       endstop.
      */
1004
1005 /** @brief STEP pin of the driver */
1006 const int stepPin = 2;
1007
1008 /** @brief Direction pin of the driver */
1009 const int dirPin = 5;
1010
1011 /** @brief Enable pin of the driver */
1012 const int enPin = 8;
1013
1014 /** @brief Endstop input pin */
1015 const int endstopPin = 9;
1016
1017 /** @brief Step delay in microseconds */
1018 const int stepDelayUs = 1000;
1019
1020 /** @brief Number of steps for the backoff movement */
1021 const int backoffSteps = 50;
1022
1023 /** @brief Initializes the hardware and starts the homing routine */
1024 void setup() {
1025     pinMode(stepPin, OUTPUT);
1026     pinMode(dirPin, OUTPUT);
1027     pinMode(enPin, OUTPUT);
1028     pinMode(endstopPin, INPUT_PULLUP);
1029
1030     Serial.begin(115200);
1031
1032     digitalWrite(enPin, LOW);
1033     digitalWrite(dirPin, LOW);
1034
1035     homeAxis();
1036 }
1037
1038 /** @brief Main loop (unused in this example) */
1039 void loop() {
1040 }
1041
1042 /** @brief Executes a single motor step */
1043 void oneStep() {
1044     digitalWrite(stepPin, HIGH);
1045     delayMicroseconds(stepDelayUs);
1046     digitalWrite(stepPin, LOW);

```

```
1048     delayMicroseconds(stepDelayUs);
1050 }
1051 /**
1052  * @brief Moves the motor by a defined number of steps
1053  * @param steps Number of steps to execute
1054  * @param dir Direction of movement (LOW/HIGH)
1055  */
1056 void moveSteps(int steps, bool dir) {
1057     digitalWrite(dirPin, dir);
1058     for (int i = 0; i < steps; i++) {
1059         oneStep();
1060     }
1061 }
1062 /** @brief Performs the homing sequence and backoff movement */
1063 void homeAxis() {
1064     while (digitalRead(endstopPin) == HIGH) {
1065         oneStep();
1066     }
1067     delay(200);
1068     moveSteps(backoffSteps, HIGH);
1069 }
1070 }
```

../Code/referenzfahrt/referenzfahrt.ino

1.12. Further Readings

- Zielke, Dirk: *Mikrosysteme*. Springer, 2025. [Zie25] Das Buch behandelt die Grundlagen und Anwendungen von Mikrosystemen und bietet einen Überblick über Aufbau, Funktion und Einsatz moderner Sensorsysteme.
- Tränkler, Hans-Rolf und Reindl, Leo: *Sensortechnik*. Springer, 2014. [TR14] Das Buch behandelt grundlegende Prinzipien der Sensortechnik und beschreibt verschiedene Sensorarten sowie deren Einsatz in technischen Systemen.

2. Druckknopf

2.1. General

Ein elektrischer Druckknopf, auch Drucktaster genannt, ist ein elektromechanisches Bedienelement, mit dem ein Stromkreis durch Drücken geöffnet oder geschlossen wird. Je nach Ausführung kann er als Schließer oder Öffner arbeiten. Nach dem Loslassen kehrt ein Taster meist wieder in seine Ausgangsstellung zurück. [RS:2026]

2.1.1. Functions

Irgendwelche Extra Funktionen? Oder Bibs

2.2. Example

Eine LED geht an bzw aus.

2.2.1. Manual

2.2.2. Code

2.2.3. Files

2.3. Einfacher Testcode

Das Programm gibt 0 oder 1 aus

2.3.1. Manual

2.3.2. Code

2.4. Einfache Anwendung

Interrupt? Für Pause oder Start und mehrere LEDS gehen nach einander an.

2.4.1. Manual

2.4.2. Code

2.5. Further Readings

Teil III.

Domain Knowledge - Tools

Teil IV.

Development

Teil V.

Monitoring

Teil VI.

**Verification - Evaluation -
Conclusion**

Teil VII.

Anhang

Literaturverzeichnis

- [3dj] *Creality Endschalter*. 2026. URL: <https://www.3djake.de/creality-3d-drucker-ersatzteile/endschalter> (besucht am 26.04.2026).
- [Arda] *Digital Pins - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/variablen/constants/inputOutputPullup/> (besucht am 25.04.2026).
- [Ardb] *Serial.print() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/communication/serial/print/> (besucht am 25.04.2026).
- [Ardc] *delay() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/time/delay/> (besucht am 25.04.2026).
- [Ardd] *digitalRead() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/digitalread/> (besucht am 25.04.2026).
- [Arde] *loop() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/loop/> (besucht am 25.04.2026).
- [Ardf] *pinMode() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/pinmode/> (besucht am 25.04.2026).
- [Ardg] *setup() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/setup/> (besucht am 25.04.2026).
- [Bas] *Creality Endschalter*. 2026. URL: <https://www.bastelgarage.ch/creality-endschalter-limit-switch> (besucht am 26.04.2026).
- [Ber24] H. Bernstein. *Messelektronik und Sensoren. Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. 2. Aufl. Springer Vieweg Wiesbaden, 25. Jan. 2024. ISBN: 978-3-658-38929-1. DOI: <https://doi.org/10.1007/978-3-658-38929-1>.
- [Hel] W. Helbig. *Praxiswissen in der Messtechnik. Arbeitsbuch für Techniker, Ingenieure und Studenten*.
- [Her+12] E. Hering u. a. *Elektrotechnik und Elektronik für Maschinenbauer*. Bd. 2. Springer, 14. Feb. 2012. ISBN: 978-3-642-12881-3.
- [TR14] H.-R. Tränkler und L. Reindl, Hrsg. *Sensortechnik: Handbuch für Praxis und Wissenschaft*. 2. Aufl. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, S. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: [10.1007/978-3-642-29942-1](https://doi.org/10.1007/978-3-642-29942-1).
- [Zie25] D. Zielke. *Mikrosysteme*. Bd. 1. Springer, 5. März 2025. ISBN: 978-3-662-71232-0.

